

MCP Development Guidelines for SDLC, DevOps & Release Management

Clear Engineering Standards for Building, Testing, and Releasing MCP-Enabled Agentic AI Systems

Prepared by	Enterprise Architecture Team
Audience	Development Teams, DevOps Engineers, Release Managers, QA Leads
Classification	Internal — Restricted Distribution
Primary Reference	AWS Prescriptive Guidance: MCP Strategies on AWS (March 2026)
Supporting Reference	NSA CSI: MCP Security Design Considerations (May 2026)
Version	1.0 03 June 2026

Table of Contents

1	Purpose & Scope	3
2	Governing Principles	3
3	MCP Tool Design Standards	4
3.1	Tool Scoping Rules	4
3.2	Tool Definition Standards	5
3.3	Tool Naming & Organisation	5
4	SDLC Integration — Phase-by-Phase Requirements	6
4.1	Phase 1: Requirements & Design	6
4.2	Phase 2: Development	7
4.3	Phase 3: Testing	8
4.4	Phase 4: Release & Deployment	9
4.5	Phase 5: Operations & Monitoring	10
5	DevOps Pipeline Standards	11
6	Testing Strategy	12
6.1	Testing Pyramid for MCP	12
6.2	Golden Dataset Requirements	13
6.3	Tool Selection Accuracy Metrics	13
7	Release Management & Quality Gates	14
8	Hosting & Deployment Standards	15
9	Governance, Auth & Compliance	16
10	Appendix: Mandatory Checklists	17

1. Purpose & Scope

This document establishes mandatory Enterprise Architecture guidelines for all development teams building, integrating, or operating Model Context Protocol (MCP) components. It covers the full software development lifecycle (SDLC) from design through production operations, with specific requirements for DevOps pipeline integration, testing, and release management.

Scope: These guidelines apply to any team that:

- Builds or consumes MCP servers (custom or open-source)
- Embeds MCP in agentic AI pipelines, LLM applications, or automation workflows
- Operates remote or gateway-hosted MCP infrastructure
- Is responsible for the release and quality assurance of MCP-dependent services

Non-Negotiable

No MCP server — custom-built or third-party — may be deployed to a production environment without passing the quality gates defined in Section 7 and receiving an approved EA Architecture Review Board (ARB) sign-off.

2. Governing Principles

All MCP development and operations must be aligned to five governing principles, derived from the AWS Well-Architected Framework as applied to MCP deployments:

Security First	Token isolation, scoped-down credentials, separate read/write authorisation, and mandatory audit logging at every MCP boundary. Security is never optional or deferred.
Operational Excellence	Tool selection accuracy metrics, golden datasets for regression testing, and structured observability. Every deployment must be measurable and improvable.
Reliability	Per-user and per-tool rate limiting, load shedding, circuit breakers, and deterministic tool execution wherever feasible.
Performance Efficiency	Workflow-scoped tools, tool filtering, and semantic search to reduce context window consumption. Tool count and definition verbosity directly impact LLM accuracy and cost.
Cost Optimisation	Reusable MCP servers across teams, shared governance infrastructure, and reduced per-request token costs through disciplined tool filtering and registration.

3. MCP Tool Design Standards

Tool design is the highest-leverage decision in any MCP implementation. How tools are scoped, named, and documented directly determines LLM accuracy, context window consumption, and maintenance cost. The following standards are mandatory for all custom MCP tool development.

3.1 Tool Scoping Rules

Tools must be scoped to **complete user workflows**, not raw API endpoints. Teams must choose between granular (per-API) and coarse-grained (workflow-oriented) approaches based on the complexity and intent of the target workflow.

Criterion	Granular (Per-API)	Coarse-Grained (Workflow)	EA Mandate
Use when	Single discrete actions, simple reversible operations	3+ sequential API calls required, complex orchestration	Workflow drives the choice
LLM calls	One per API step — high latency	Single call per workflow — lower latency	Prefer coarse-grained
Context cost	Higher — each tool definition consumes tokens	Lower — fewer tools registered	Minimise token use
Error handling	Agent must coordinate — fragile	Deterministic inside tool — robust	Prefer deterministic
Max params	8 or fewer	8 or fewer; if bundling requires more, bundling wins	Hard limit: 8

Mandatory scoping rules:

- **Read/write separation is mandatory.** Provide distinct tools for read and write operations. This enables independent authorisation policies and reduces unintended data modification risk.
- **Bundle if 3+ APIs commonly co-occur.** If agents repeatedly chain three or more tools to complete a workflow, wrap them in a single coarse-grained tool.
- **Decompose if exceeding 8 parameters or 2 distinct intents.** A tool attempting to handle multiple user intents simultaneously confuses LLM parameter selection.
- **Prefer sensible defaults.** The LLM should specify only request-specific parameters; defaults handle everything else.
- **Use enums for finite value sets** (status, priority, category). Use descriptions with examples for free-form fields.
- **Test with real user prompts** before any tool definition is considered complete. Accuracy must be verifiable.

3.2 Tool Definition Standards

All tools must follow the MCP tool specification. The **tool specification approach** is the EA-mandated standard. Docstring-only definitions are acceptable *only* for Python implementations where an organisation-wide docstring standard has been formally approved and documented.

Required Fields for Every Tool

- **name** — Follows the domain-noun-verb naming standard (see 3.3)
- **description** — Written as a prompt: purpose (what), trigger condition (when), output context (what the result is used for), and error conditions
- **inputSchema** — Full JSON Schema with descriptions per field; enums where applicable; examples for free-form fields
- **outputSchema** — Defined output structure so downstream consumers know what to expect
- **Concrete workflow examples** — At minimum one end-to-end example with real values, embedded in the description

3.3 Tool Naming & Organisation

Tools must follow the **domain-noun-verb** naming pattern. This prevents name collisions, enables alphabetical clustering of related tools, and facilitates both LLM tool scanning and human navigation.

Pattern	Correct Example	Incorrect Example	Reason
domain-noun-verb	github_issue_create	createGitIssue	camelCase breaks clustering; no domain prefix
domain-noun-verb	github_pullrequest_merge	mergePR	Ambiguous; no domain or noun
domain-noun-verb	db_customer_read	getCustomer	No domain; verb before noun
domain-noun-verb	s3_object_upload	uploadToS3	Action before resource; domain suffix only

Server organisation rules:

- **Hard cap: 50 tools per MCP server.** Beyond 50, split into multiple domain-bounded servers. Consumers may naively load all tools — high counts silently degrade LLM accuracy.
- **Separate servers for read and write operations** where separation of duties is required (e.g., database read replica vs. write master).
- **Use conditional server loading** at the agent level: load write-access servers only when the detected user intent requires mutation.
- **Register all production servers** in the enterprise MCP Registry before deployment.

4. SDLC Integration — Phase-by-Phase Requirements

Every MCP component must traverse a defined SDLC process. The following phases are mandatory for all custom MCP server development and for any third-party MCP server integrated into enterprise systems.

Phase 1

Requirements & Design

- **Define tool scope and user workflow map** before writing any code. Document each target workflow as a user story; identify the minimum viable toolset.
- **EA ARB Design Review required** for any MCP server that: (a) accesses regulated data (PII, HIPAA, financial), (b) exposes write operations, or (c) is intended for shared/multi-team use.
- **Evaluate existing MCP servers first.** Document the evaluation outcome: sufficient as-is, requires enhancement, or requires custom build. Custom builds require explicit justification.
- **Define data classification** for all data the MCP server will access. Map each tool to its classification zone (public, internal, confidential, restricted).
- **Authentication pattern selection** (machine-to-machine vs. user-delegated) must be documented in the design artefact and approved before development begins.
- **Tool definition drafts** including name, description, inputSchema, and outputSchema must be peer-reviewed before implementation.

Phase 2

Development

- **Follow the tool design standards in Section 3** without exception. Non-compliant tool definitions will be rejected at code review.
- **Implement read/write tool separation** as separate functions; never combine in a single tool handler.
- **Token isolation is mandatory:** each tool must acquire its own scoped access token for downstream resources. Propagating the caller's token is a blocking defect.
- **Embed compliance logic deterministically** for regulated workflows (e.g., HIPAA validation, PII anonymisation) inside the tool implementation — not in LLM orchestration.
- **Implement circuit breakers and retry logic** inside coarse-grained tool implementations. The LLM layer must not be responsible for retry orchestration.
- **Rate limiting must be implemented** at the tool level: per-user and per-tool limits are mandatory for any server exposed to more than one consumer.
- **All tools must be unit testable in isolation** — tool handlers must accept injected dependencies (HTTP clients, DB connections) to enable mocking.
- **Structured logging must be implemented** at every tool invocation: log tool name, parameters (sanitised), identity, timestamp, and result hash.

Phase 3

Testing

- **Unit tests:** Every tool handler must have unit tests with mocked dependencies achieving minimum 80% line coverage.
- **Integration tests:** Tools must be tested against real (or realistic sandbox) downstream services in an isolated test environment.
- **Tool selection accuracy tests:** Run golden dataset prompts (see Section 6.2) against the registered tool set; measure and record the tool selection accuracy score. Minimum threshold: 85% accuracy.
- **Regression suite:** Any change to a tool name, description, or schema triggers a full regression run against the golden dataset.
- **Adversarial / injection tests:** Test tool parameter inputs with malformed, oversized, and injection-attempt payloads. All must be rejected gracefully without exposing errors.
- **Load and rate-limit tests:** Validate rate limiting behaviour under simulated peak load; confirm DoS mitigation (prompt storms, recursive task requests) does not degrade adjacent services.
- **Auth and token tests:** Validate token scoping, expiry, and rejection of replayed tokens. Confirm that write tools reject read-only tokens.

Phase 4

Release & Deployment

- **All quality gates in Section 7 must be passed** and recorded before any production deployment is authorised.
- **MCP Registry registration** is mandatory before production deployment. Include: server name, version, owner team, data classification, tool list, and known limitations.
- **Semantic versioning is required:** MAJOR version increments when tool names, schemas, or authentication requirements change in a breaking manner.
- **Deprecation policy:** Tools must not be removed without a minimum 90-day deprecation period during which both old and new versions are served. Agents must be migrated before removal.
- **Deployment via CI/CD pipeline only** — no manual production deployments. All MCP server deployments must pass automated pipeline gates.
- **Blue/green or canary deployment** is the required strategy for any MCP server with active consumers. Full cutover requires observed error rate below 0.5% over a 30-minute window.
- **Rollback plan must be documented and tested** before each production release.

Phase 5

Operations & Monitoring

- **SIEM integration is mandatory:** all tool invocation logs must flow to the enterprise SIEM within 5 minutes of occurrence.
- **Alerting thresholds must be configured** at deployment time: authorisation failure spikes, unexpected tool invocations, error rate thresholds, and latency SLA breaches.
- **Operational metrics dashboard** must be maintained per server: tool invocation volume, tool selection accuracy trend, p50/p95/p99 latency, error rates by tool.
- **Vulnerability scanning:** MCP servers must be included in the enterprise vulnerability scanning cadence. Critical CVEs require patch within 72 hours; High within 2 weeks.
- **Quarterly capability review:** review registered tool definitions for accuracy, relevance, and performance. Archive tools that have not been invoked in 90+ days.
- **Patch and update policy:** all consumers of a shared MCP server must be notified 14 days before a breaking change is applied.

5. DevOps Pipeline Standards

All MCP server builds must run through a CI/CD pipeline that enforces the following stage requirements. No stage may be skipped or overridden without documented EA exception approval.

PR / Commit

- **Automated lint:** tool naming convention enforced (domain-noun-verb); reject non-conforming names.
- **Schema validation:** inputSchema and outputSchema validated against JSON Schema draft-07 minimum.
- **Static analysis:** SAST scan for injection vulnerabilities (CWE-77, CWE-78, CWE-94); blocking on HIGH/CRITICAL findings.
- **Unit test run:** 80% line coverage gate; no skipped tests permitted without annotation.
- **Dependency check:** SCA scan against CVE database; block on CRITICAL severity in direct dependencies.

Build / Package

- **Container image build** with pinned base image digest — no 'latest' tags in production images.
- **SBOM generation** (Software Bill of Materials) attached to every build artefact.
- **Image scanning:** container vulnerability scan (e.g., Amazon Inspector, Trivy); block on CRITICAL findings.
- **Secrets scanning:** confirm no credentials, API keys, or tokens embedded in image or configuration files.

Integration Test

- **Integration test suite** runs against a dedicated test environment with real (sandboxed) downstream dependencies.
- **Golden dataset evaluation:** tool selection accuracy score computed and stored. Failure to meet 85% threshold blocks promotion.
- **Rate limit validation:** confirm per-user and per-tool limits function correctly under simulated concurrent load.
- **Auth flow validation:** token isolation, scope enforcement, and token expiry all exercised and verified.

Staging Deploy & Smoke

- **Deploy to staging** using identical infrastructure-as-code as production (no staging-specific shortcuts).
- **Automated smoke test:** invoke every registered tool with a valid prompt; assert successful response within SLA.
- **MCP Registry pre-registration:** staging entry created and validated before production promotion is approved.
- **Performance baseline:** capture p50/p95/p99 latency baselines; compare against previous version; flag regressions > 15%.

Production Deploy

- **EA ARB sign-off** recorded in the release tracking system before production deployment initiates.
- **Canary or blue/green deployment** — no big-bang releases. Canary at 5% traffic minimum 15 minutes before ramp.
- **Automated rollback trigger:** error rate > 1% or p99 latency > 2x baseline automatically rolls back.
- **Post-deploy verification:** smoke test re-executed against production; SIEM alert rules confirmed active.
- **MCP Registry updated** to production status with version, changelog, and owner contact.

6. Testing Strategy

6.1 Testing Pyramid for MCP

Level	Type	What Is Tested	Tooling (Examples)	Coverage Gate
L1 — Unit	Unit Tests	Individual tool handler functions with mocked dependencies	pytest, Jest, JUnit	80% line coverage minimum
L2 — Component	Integration Tests	Tool end-to-end against real sandbox downstream services	Testcontainers, LocalStack, WireMock	100% of tools exercised
L3 — Contract	Schema Contract Tests	inputSchema and outputSchema correctness; backward compat on change	Pact, Dredd, custom JSON Schema validators	Zero schema violations
L4 — Accuracy	Tool Selection Accuracy	LLM selects the correct tool for golden dataset prompts	Custom eval harness; Amazon Bedrock eval	≥ 85% accuracy required
L5 — Security	Adversarial / Injection	Malformed inputs, injection payloads, oversized parameters, replay attacks	OWASP ZAP, custom fuzz harness	Zero critical/high failures
L6 — Load	Performance & Rate Limit	Behaviour under sustained load, prompt storms, concurrent users	k6, Locust, Artillery	Rate limits enforced; no cascade failure

6.2 Golden Dataset Requirements

Every MCP server must maintain a **golden dataset**: a versioned set of representative user prompts paired with the expected tool invocation and parameters. This dataset is the regression baseline for all tool definition changes.

- **Minimum 50 prompts per MCP server** at initial release; minimum 10 additional prompts per major tool addition.
- **Coverage requirement:** every registered tool must be the correct answer for at least 3 prompts in the golden dataset.
- **Ambiguous-intent prompts required:** include at least 10 prompts where two tools could plausibly be invoked; validate LLM selects the higher-intent match.
- **Negative prompts required:** include prompts that should result in no tool invocation or an explicit refusal; validate no hallucinated tool calls occur.
- **Versioning:** golden dataset is versioned alongside the MCP server. Dataset changes require peer review and ARB notification.
- **Automated execution:** the golden dataset must be runnable as a CI pipeline step without manual intervention.

6.3 Tool Selection Accuracy Metrics

Tool selection accuracy is tracked as a primary quality metric. The following measurements must be captured at every pipeline run and stored in the team's observability platform:

Metric	Definition	Minimum Threshold	Alert Threshold
Tool Selection Accuracy	% of golden dataset prompts where the LLM selects the correct tool	85%	< 80%

Metric	Definition	Minimum Threshold	Alert Threshold
Parameter Accuracy	% of correct tool invocations where all required parameters are correctly populated	90%	< 85%
False Tool Invocation Rate	% of prompts where an incorrect tool is invoked	< 10%	> 15%
Hallucination Rate	% of prompts where a non-existent tool is called	0%	> 1%
Regression Delta	Change in tool selection accuracy vs. previous version baseline	0% or improvement	Decline > 3%

7. Release Management & Quality Gates

All MCP server releases must pass the following quality gates in sequence. Gates are strictly ordered — a failing gate blocks all subsequent stages. Gate outcomes must be recorded in the release tracking system.

Gate	Stage	Owner	Mandatory Criteria	Blocking?
G1	Code Review	Tech Lead + EA Rep	Tool naming compliant; read/write separation enforced; token isolation confirmed; structured logging present; no hardcoded credentials	YES
G2	Security Scan	Security Team	Zero CRITICAL/HIGH SAST findings; zero CRITICAL container CVEs; no embedded secrets	YES
G3	Unit Test	CI System	80% line coverage; zero failing tests; no skipped tests without annotation	YES
G4	Schema Contract	CI System	inputSchema and outputSchema validate against JSON Schema; no breaking changes without MAJOR version bump	YES
G5	Integration Test	CI System	100% of tools exercised; all auth flows pass; rate limiting enforced	YES
G6	Accuracy Eval	CI System + QA Lead	Golden dataset tool selection accuracy $\geq 85\%$; parameter accuracy $\geq 90\%$; hallucination rate = 0%	YES
G7	Load Test	Platform / SRE	Rate limits hold under 2x expected peak; no cascade failure; p99 latency within SLA	YES
G8	Staging Smoke	QA Lead	All tools return valid responses; MCP Registry staging entry validated	YES
G9	ARB Sign-off	EA ARB	Design artefact reviewed; data classification confirmed; auth pattern approved; Registry entry complete	YES
G10	Production Deploy	Release Manager	Canary deployed; automated rollback triggers confirmed active; SIEM alerts live; post-deploy smoke passed	YES

Release versioning policy:

Version Type	When Required	Notification Required	Deprecation Period
PATCH (x.y.Z)	Bug fixes, description improvements, no schema change	None required	N/A
MINOR (x.Y.0)	New tools added; non-breaking schema additions	14-day notice to consumers	N/A
MAJOR (X.0.0)	Tool removed or renamed; breaking schema change; auth change	90-day deprecation; parallel versions maintained	90 days minimum

8. Hosting & Deployment Standards

Teams must select a hosting model that matches the operational maturity, consumer count, and data classification of their MCP server. The following matrix defines the approved hosting patterns and their mandatory requirements.

Hosting Model	Approved For	Not Approved For	Mandatory Controls
Local	Developer tooling, IDE integrations, single-engineer workflows, rapid prototyping	Any multi-user, regulated data, or production agentic system	Must be registered in enterprise MCP Registry; versioning enforced; security scanning before distribution
Remote (HTTP/S)	Shared team tools, production agentic systems, multi-tenant access	Deployments without centralised auth and identity management	Authentication mandatory; token isolation per tool; TLS enforced; rate limiting; structured logging to SIEM
MCP Gateway	Large multi-server ecosystems, cross-team tool discovery, dynamic tool registration	Simple single-server deployments where gateway adds unnecessary complexity	All remote controls plus: semantic tool search enabled; auth and routing policies enforced at gateway; single audit log stream

Deployment rules applicable to all hosting models:

- **Infrastructure as Code only:** all MCP server infrastructure must be defined and deployed via IaC (e.g., Terraform, CloudFormation, CDK). No click-ops in staging or production.
- **Immutable deployments:** deployed container images are never modified in-place. All changes produce a new versioned image.
- **Sandbox for context-window optimisation:** when deploying to remote or gateway, implement tool filtering or semantic search to minimise context window consumption per agent request.
- **Local server distribution:** local MCP servers distributed to developer machines must go through the enterprise MCP Registry with security scanning and approval workflow — not shared via ad hoc channels.

9. Governance, Authentication & Compliance

9.1 Authentication & Authorisation Requirements

Requirement	Machine-to-Machine	User-Delegated Access
Token source	Pre-provisioned service identity; purpose-scoped token per tool	OAuth 2.1 + PKCE; user consent flow; token scoped to specific tool actions
Token propagation	PROHIBITED — do not pass caller tokens to downstream services	PROHIBITED — MCP server generates a new scoped token for each downstream call
Token lifecycle	Short-lived (max 1 hour); rotation automated; revocation list maintained	Short-lived; bound to user session; revocable on session termination
Audience validation	Validate 'aud' claim matches receiving server on every token use	Validate 'aud' claim; confirm scope covers the requested operation
Logging	Log every token issuance and use with tool identity and timestamp	Log user identity, consent event, tool invocation, and result classification

9.2 Operational Metrics & Governance

Required operational metrics per MCP server:

- **Tool invocation count** — per tool, per user identity, per time window
- **Tool selection accuracy trend** — tracked weekly; decline > 3% triggers review
- **Authorisation failure rate** — alert if > 0.5% of invocations; investigate > 1%
- **Token issuance and revocation events** — full audit trail in SIEM
- **p50 / p95 / p99 latency per tool** — SLA: p99 < 3 seconds for standard tools
- **Error rate by tool and error type** — distinguish client errors (4xx) from server errors (5xx)
- **Rate limit hit rate** — track which users and tools are hitting limits; inform capacity planning

9.3 Compliance Requirements by Data Classification

Data Class	Examples	Additional Requirements
Public	Weather data, public documentation	Standard controls; MCP Registry registration; basic logging
Internal	Internal APIs, team databases, code repositories	Auth required; token isolation; audit logging; ARB review
Confidential	Customer data, financial records, proprietary IP	All internal controls + DLP integration; data residency controls; quarterly access review
Restricted / Regulated	PII, HIPAA-covered health data, payment card data	All confidential controls + compliance validation embedded in tool (not LLM orchestration); anonymisation before processing; legal/compliance sign-off on tool design

10. Appendix: Mandatory Checklists

Checklist A — Pre-Development (Design Phase)

- User workflow map completed and approved
- Existing MCP server evaluation documented (sufficient / enhance / custom build)
- Data classification assigned to all data accessed by this server
- Authentication pattern selected and documented (M2M or user-delegated)
- Tool scope decisions recorded (granular vs. coarse-grained justification)
- Tool definition drafts (name, description, inputSchema, outputSchema) peer-reviewed
- EA ARB design review scheduled (required if: regulated data / write operations / shared server)
- MCP Registry entry draft created

Checklist B — Development Completion (Pre-Test)

- All tool names follow domain-noun-verb pattern
- Read and write operations implemented as separate tools
- Token isolation implemented: each tool acquires its own scoped downstream token
- No caller tokens propagated to downstream services
- Compliance logic embedded deterministically in tool (not LLM orchestration) for regulated workflows
- Circuit breakers and retry logic implemented inside coarse-grained tools
- Per-user and per-tool rate limiting implemented
- Structured logging implemented at every tool invocation
- All tool handlers accept injected dependencies (unit-testable)
- No hardcoded credentials, API keys, or secrets in source code or configuration
- Tool count per server is 50 or fewer
- Parameter count per tool is 8 or fewer (or justified exception documented)

Checklist C — Testing Completion (Pre-Release)

- Unit tests: 80%+ line coverage achieved
- Integration tests: 100% of tools exercised against sandbox dependencies
- Schema contract tests: zero violations
- Golden dataset: minimum 50 prompts covering all tools
- Tool selection accuracy: $\geq 85\%$ on golden dataset
- Parameter accuracy: $\geq 90\%$
- Hallucination rate: 0% (no invocations of non-existent tools)
- Adversarial / injection tests: zero CRITICAL or HIGH failures

■	Load test: rate limits hold at 2x expected peak
■	Auth tests: token scoping, expiry, and replay rejection all verified
■	SAST scan: zero CRITICAL/HIGH findings
■	Container image scan: zero CRITICAL CVEs
■	Secrets scan: clean

Checklist D — Release Readiness (Pre-Production)

■	All quality gates G1–G9 passed and recorded in release tracking system
■	MCP Registry entry complete: server name, version, owner, tools list, data classification
■	Semantic versioning applied correctly (PATCH / MINOR / MAJOR)
■	Consumers notified per versioning notification policy
■	Rollback plan documented and tested in staging
■	Blue/green or canary deployment strategy confirmed
■	Automated rollback trigger thresholds configured
■	SIEM alert rules created and validated
■	Operational dashboard configured with all required metrics
■	EA ARB sign-off recorded
■	Post-deploy smoke test checklist prepared

Checklist E — Third-Party / Open-Source MCP Server Intake

■	Maintenance status verified (not archived; actively maintained)
■	Source code reviewed using most stringent code review profile
■	Security scan performed (SAST + dependency CVE check)
■	Tool definitions reviewed against EA naming and definition standards
■	Authentication and token handling reviewed and documented
■	Data classification impact assessed
■	Enterprise MCP Registry entry created before any integration begins
■	Approved for local-only or remote deployment (justified and documented)
■	Known limitations and open security issues documented for consumers
■	Quarterly re-evaluation reminder scheduled

References: AWS Prescriptive Guidance — Model Context Protocol Strategies on AWS (March 2026) | NSA CSI — MCP Security Design Considerations (May 2026) | OWASP ASVS V7 Session Management | AWS Well-Architected Framework
Enterprise Architecture Team | Version 1.0 | 03 June 2026 | Internal — Restricted Distribution